Stats 21 - HW 3

Fiona Xu

Homework copyright Miles Chen. The word problems are adapted from exercises in *Think Python*, 2nd edition, by Allen B. Downey.

Complete the code and written responses. Before submitting, restart the kernel and run your completed notebook from top to bottom. Make sure all requested results are visible. Either `print(...)` or a visible notebook result is acceptable.

Export to HTML, print to a letter-sized PDF, and submit the PDF to Gradescope. Assign/tag the pages to the corresponding questions. Follow the course submission instructions for the completed notebook file as well.

Academic Integrity

The homework is practice for building your own understanding. Work through the problems, check your reasoning, and use permitted help to support your learning.

Replace `[Your Name]` to acknowledge the following:

> I, Fiona Xu, agree to follow the course academic integrity policies and the Stats 21 AI and Collaboration Policy. My submitted work reflects my own thinking and understanding, and I can explain my code and results.

Acknowledgments and AI disclosure: Give specific credit for people or sources you relied on, and disclose any AI assistance, including hints, debugging, or editing. Briefly state which problems and how each helped, or write "None".

> Referenced class notes for almost every question. Used Claude to debug NP.2b, specifically to link two functions together using " | ".

Reading and class connections

Use *Think Python*, **3rd edition**, for the reading:

- Chapter 10: Dictionaries.
- Chapter 11: Tuples.

- Chapter 13: Files and Databases, focusing on text-file reading and writing; databases are not required here.

The exercise numbers and source links below refer to the **2nd edition**. The full prompts are included, so you do not need a separate copy of that edition.

Review lectures 8–10 for dictionaries, tuples, and files, and lectures 11–12 for NumPy arrays, copies/views, broadcasting, and Boolean masks.

Working on this homework

- Return the requested result from each function. Display results in the supplied test cells after completing the function. Keep the tests and add cases where requested.
- You may assume inputs meet the stated requirements; extra input-validation code is not required. Use clear intermediate steps; one-line solutions earn no extra credit.
- Use a dictionary for the word problems and NumPy array operations for the NumPy section. A solution should show the requested method, not hard-code the displayed output.
- Written checks normally need only one or two sentences.

`words.txt` is included beside this notebook in `homework/HW3`; it is the same word list used in HW2. Run this notebook with `HW3` as the working directory, or adjust the file path if you keep it elsewhere. Do not print the entire word dictionary.

Exercise 11.1: A dictionary of words

Adapted from Think Python 2e, Exercise 11.1.

Complete `make_word_dict(filename="words.txt")`. Read the file and **return** a dictionary whose keys are the words, with surrounding whitespace removed. Skip blank lines. You may use `None` for every value. The file contains lowercase words.

Save the returned dictionary as `word_dict`. Display its number of keys and the results of the membership checks below.

Understanding: Does `"hello" in word_dict` search keys or values? Why should we remove the newline before using a word as a key?

In [1]:
```
def make_word_dict(filename="words.txt"):
    word_dict = {}
    file = open(filename)
    for line in file:
        word = line.strip()
        if word != "":
            word_dict[word] = None
    return word_dict
```

```
word_dict = make_word_dict()
print(len(word_dict))
print("hello" in word_dict)
```

```
113776
True
```

In [2]:
```
# Run after completing make_word_dict().
word_dict = make_word_dict()
print("Number of words:", len(word_dict))
print("hello" in word_dict)       # Expected: True
print("hello\n" in word_dict)   # Expected: False
print("" in word_dict)            # Expected: False
```

```
Number of words: 113776
True
False
False
```

Your explanation: Searching for something in a dictionary with 'in' always searches in keys, not values, so "hello" would be a key search. Removing the newline character removes it from being part of the key, leaving only the word (string).

Exercise 11.4: Detecting duplicates with a dictionary

Adapted from Think Python 2e, Exercise 11.4.

Rewrite HW2's `has_duplicates(t)` using a dictionary to keep track of values already encountered. Return `True` if any value repeats, and `False` otherwise. Do not modify the input list, sort it, or replace the dictionary with a set.

As in HW2, inputs may mix integers and strings and will not contain nested lists. All values used here are hashable and can be dictionary keys. An empty list has no duplicates.

Understanding: What information does your dictionary represent as you traverse the list, and why can you return as soon as you find a repeat? No timing experiment or claim of a fixed speedup is required.

In [3]:
```
def has_duplicates(t):
    bool_dict = {}
    for word in t:
        if word in bool_dict:
            return True
        bool_dict[word] = None
    return False
```

In [4]:
```
print(has_duplicates(['a', 'b', 'b', 'c']))  # Expected: True
print(has_duplicates(['a', 'b', 'c', 'a']))  # Expected: True
print(has_duplicates(['a', 'b', 'c']))       # Expected: False
```

```
print(has_duplicates([]))                    # Expected: False
print(has_duplicates([1, '1', 2]))           # Expected: False

values = [3, 1, 3]
before = values.copy()
print(has_duplicates(values))                # Expected: True
print(values == before)                      # Expected: True
# Add one more test of your own and state its expected result.

print(has_duplicates(['x', 'y', 'z'])) # Expected: False
```

```
True
True
False
False
False
True
True
False
```

Your explanation: As I traverse the list, my dictionary's keys represent the values I have already encountered, so checking whether an item is a key tells me whether it has appeared before. I can return `True` as soon as I find a repeat because a single repeated value is enough to show the list has a duplicate, so the remaining items don't need to be checked.

Exercise 11.5: Caesar-cipher rotate pairs

Adapted from Think Python 2e, Exercise 11.5.

A Caesar rotation shifts each letter forward through the alphabet, wrapping from `z` to `a`. For example, `"cheer"` rotated by 7 becomes `"jolly"`.

1. Complete `rotate_word(word, shift)` for lowercase `a` – `z` words and integer shifts. Return a string. A shift of 26 leaves a word unchanged.
2. Complete `find_rotate_pairs(word_dict, min_length=5)`. Use the dictionary keys as the available words. Look for **distinct** words of at least `min_length` letters related by shifts 1 through 25.
3. Return a sorted list of tuples `(word1, word2, shift)`. Report each unordered pair once, with `word1 < word2` alphabetically. `shift` must rotate `word1` into `word2`. Do not also report the reverse pair or a word paired with itself.
4. Search the full word dictionary and display the **total number of pairs**. Then display only the **first 10 pairs from the sorted result**, one per line. Compute and return all pairs; limit only the displayed examples.

Hints: `ord()` converts a character to its numeric code and `chr()` converts a code back to a character. Modulo 26 can handle wraparound. Generate rotations and use dictionary membership instead of comparing every word with every other word.

Understanding: Why are shifts 1–25 sufficient for finding distinct rotate pairs, and how does your output rule avoid counting a pair twice?

In [5]:
```
def rotate_word(word, shift):
    string = ""
    for letter in word:
        offset = (ord(letter) - ord("a") + shift) % 26
        string += chr(ord("a") + offset)
    return string


def find_rotate_pairs(word_dict, min_length=5):
    pairs = []
    for word in word_dict:
        if len(word) >= min_length:
            for shift in range(1, 26):
                rotated = rotate_word(word, shift)
                if rotated in word_dict and word < rotated:
                    pairs.append((word, rotated, shift))
    return sorted(pairs)
```

In [6]:
```
print(rotate_word("cheer", 7))  # Expected: jolly
print(rotate_word("zoo", 1))    # Expected: app
print(rotate_word("abc", 26))   # Expected: abc

small_words = {"cheer": None, "jolly": None, "melon": None, "a": None}
print(find_rotate_pairs(small_words))  # Expected: [('cheer', 'jolly', 7)]
# Add one more test of your own, including its expected result.
test_words = {"aptly": None, "timer": None, "arena": None, "a": None}
print(find_rotate_pairs(test_words))  # Expected: [('aptly', 'timer', 19)]
```

```
jolly
app
abc
[('cheer', 'jolly', 7)]
[('aptly', 'timer', 19)]
```

In [7]:
```
# Find all pairs from word_dict; display the total count and the first 10 sc

pairs = find_rotate_pairs(word_dict)
print(len(pairs))
for pair in pairs[:10]:
    print(pair)
```

```
97
('abjurer', 'nowhere', 13)
('adder', 'beefs', 1)
('ahull', 'gnarr', 6)
('alkyd', 'epoch', 4)
('alula', 'tenet', 19)
('ambit', 'setal', 18)
('anteed', 'bouffe', 1)
('aptly', 'timer', 19)
('arena', 'river', 17)
('baiza', 'tsars', 18)
```

Your explanation: Shifting 1-25 is sufficient because a shift of 26 would just loop back to the original character, and anything above 26 would be the same as 26 + any value from 1-25. My output avoids counting the same word twice by using the word < rotated line, since only one word of the two can come before the other.

Exercise 12.2: Grouping anagrams

Adapted from Think Python 2e, Exercise 12.2.

Anagrams use exactly the same letters with the same multiplicities. Complete `group_anagrams(words)` for an iterable of distinct lowercase words. Return a dictionary that maps a **tuple of sorted letters** to a list of its words. For example, `"eat"` and `"tea"` share the key `('a', 'e', 't')`.

Use the keys of `word_dict` as input. Identify all groups containing **at least six distinct words** and display the **total number of qualifying groups**. Alphabetize the words within each qualifying group, then order the groups by their first word. Display only the **first 10 groups** in that order, one per line. Compute all groups before limiting the display. Groups with fewer than six words should remain in the returned dictionary but need not be printed.

Example of the required group display:

`['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']`
Understanding: Why can a tuple of letters be a dictionary key while the list returned by `sorted(word)` cannot? Why must the key retain repeated letters, as in `"abb"` versus `"ab"`?

In [8]:
```
def group_anagrams(words):
    groups = {}
    for word in words:
        key = tuple(sorted(word))
        if key in groups:
            groups[key].append(word)
        else:
            groups[key] = [word]
    return(groups)
```

In [9]:
```
small_groups = group_anagrams(["eat", "tea", "ate", "bat", "tab", "abb", "ab
print(sorted(small_groups[('a', 'e', 't')]))  # Expected: ['ate', 'eat', 'te
print(sorted(small_groups[('a', 'b', 't')]))  # Expected: ['bat', 'tab']
print(small_groups[('a', 'b', 'b')])          # Expected: ['abb']
print(small_groups[('a', 'b')])               # Expected: ['ab']
```

```
['ate', 'eat', 'tea']
['bat', 'tab']
['abb']
['ab']
```

In [10]:
```
# Group the full word dictionary; display the total qualifying-group count
# and the first 10 qualifying groups in the order specified above.

anagram_groups = group_anagrams(word_dict)

qualifying = []
for key in anagram_groups:
    if len(anagram_groups[key]) >= 6:
        qualifying.append(sorted(anagram_groups[key]))

qualifying.sort()

print(len(qualifying))
for group in qualifying[:10]:
    print(group)
```

```
104
['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']
['acers', 'acres', 'cares', 'carse', 'escar', 'races', 'scare', 'serac']
['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced']
['aiders', 'deairs', 'irades', 'raised', 'redias', 'resaid']
['airts', 'astir', 'sitar', 'stair', 'stria', 'tarsi']
['alert', 'alter', 'artel', 'later', 'ratel', 'taler']
['alerting', 'altering', 'integral', 'relating', 'tanglier', 'triangle']
['alerts', 'alters', 'artels', 'estral', 'laster', 'ratels', 'salter', 'slat
er', 'staler', 'stelar', 'talers']
['alevin', 'alvine', 'valine', 'veinal', 'venial', 'vineal']
['algins', 'aligns', 'lasing', 'liangs', 'ligans', 'lingas', 'signal']
```

Your explanation: A tuple can be a dictionary key because it is immutable and therefore hashable, while the list returned by `sorted(word)` is mutable and unhashable, since its contents could change after being stored. We keep repeated letters because anagrams must match in letter counts: dropping repeats would give `"abb"` and `"ab"` the same key, `('a', 'b')`, and wrongly group them together.

NumPy exercises

Display each requested result. Use array operations and Boolean masks rather than Python loops for these exercises. Functions not demonstrated in class have documentation links below; looking up an API is part of the assignment.

Use the supplied random-number generators. A generator advances when it draws values: restart its setup cell before repeating an exercise if you want the same draws. Exact streams are not guaranteed across NumPy versions; correctness also depends on the requested shapes, ranges, and calculations, not only printed digits.

For NP.2b and NP.7, explicitly use **`ddof=0`** for standard deviation, as specified in those questions. Keep full precision for calculations unless rounding is requested.

In [11]:
```
import numpy as np
```

NP.1a: Random integers

Using the generator below, create `b`, a one-dimensional array of 10 random integers from **0 through 99 inclusive**, with repetitions allowed. Display it.

In the course environment, the first draw gives:

```
[47 51 75 95  3 14 82 94 24 31]
```

In [12]:
```
rng = np.random.default_rng(seed=1)
```

In [13]:
```
# Create and display b here.

b = rng.integers(0, 100, size=10)
print(b)
```

```
[47 51 75 95  3 14 82 94 24 31]
```

NP.1b: Reverse a slice

Use slicing with a negative step to create `b_reversed`, containing the elements of `b` in reverse order. Display both arrays without changing their values.

Understanding: Does this basic slice share data with `b`? If you assigned to an element of `b_reversed`, would `b` change? Explain using the view/copy distinction from lecture 11; you do not need to make that assignment.

In [14]:
```
# Create and display b_reversed, then display b.

b_reversed = b[::-1]
print(b_reversed)
```

```
[31 24 94 82 14  3 95 75 51 47]
```

Your explanation: Yes they share the same data, meaning if an element of b_reversed is assigned a new value, the same element in b would also be changed. This is due to the two arrays referencing the same data/object, as b_reversed is a view of b.

NP.2a: Simulate normal observations

Using the fresh generator below, draw 1,000 values from a normal distribution with **population mean 100 and population standard deviation 15**. Round the values to one decimal place and store the resulting array as `c`.

Display only the first 10 values of `c`, followed by its length, mean, and standard deviation (`ddof=0`). Do not print all 1,000 values.

Understanding: Why need the observed mean and SD not equal 100 and 15 exactly?

In [15]:
```
rng = np.random.default_rng(seed=2)
```

In [16]:
```
# Create c and display the requested results.

c = np.round(rng.normal(loc=100, scale=15, size=1000), 1)

print(c[:10])
print(len(c))
print(c.mean())
print(c.std(ddof=0))
```

```
[102.8  92.2  93.8  63.4 127.  117.2  95.1 111.6 104.2  91.7]
1000
99.66320000000002
15.19100476466254
```

Your explanation: The observed mean and standard deviation are most likely not going to be 100 and 15 exactly because the generator generates a random sample from a normal distribution each time, not the exact population parameters.

NP.2b: A Boolean mask for extreme values

Using the rounded array `c`, compute its observed mean and SD with `ddof=0`. Select values whose **absolute distance from that mean is strictly greater than three times that SD**. Use the observed quantities, not the generating parameters 100 and 15.

Display the selected values, their count, their proportion of all 1,000 values, and the corresponding percentage. For example, 3 out of 1,000 would be a proportion of `0.003`, or `0.3%` —not a proportion of `0.3`.

Understanding: Why does taking the mean of a Boolean mask give the proportion selected? There is no requirement that this simulated sample have a particular number of extreme values.

In [17]:
```
# Compute the mask and display the values, count, proportion, and percentage

extremes = c[((c - c.mean()) > (3 * c.std(ddof = 0))) | ((c.mean() - c) > (3
print(extremes)
print(len(extremes))
print(len(extremes) / len(c))
print(len(extremes) / len(c) * 100, "%")
```

```
[145.8 145.8 146.7]
3
0.003
0.3 %
```

Your explanation: Taking the mean of a Boolean mask gives the proportion because all False values are 0 and all True values are 1, so the sum of the mask is the number of True values. Mathematically, taking the mean is equivalent to calculating the number of all

True values divided by the number of all values, which is the proportion of True values in that sample.

NP.3: An identity matrix

Create and display a 3 × 3 identity array named `I3`: ones on the main diagonal and zeros elsewhere. Its shape should be `(3, 3)`.

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [18]:
```
# Create and display I3.

I3 = np.array([1, 0, 0, 0, 1, 0, 0, 0, 1]).reshape(3, 3)
print(I3)
```

```
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

NP.4a: Reshape a sequence

Create `X`, a 10 × 10 array containing the integers 1 through 100, arranged across rows in increasing order. Display the full array and its shape. The first row should contain 1–10 and the last row 91–100.

In [19]:
```
# Create and display X and its shape.

X = np.arange(1, 101).reshape(10, 10)
print(X)
print(X.shape)
```

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
(10, 10)
```

NP.4b: Copy and mask

Make an **independent copy** of `X` named `Y`. Create an array of the ten perfect squares between 1 and 100, then use a Boolean mask to replace every nonsquare in `Y` with zero. Do not change `X`. You may prepare the squares and mask in separate lines; there is no one-line requirement.

See `np.isin` for an elementwise membership test. Display the full `Y`, and display `X` to confirm it is unchanged. `Y` should retain exactly 10 nonzero entries.

Understanding: Why would `Y = X` fail to provide an independent copy?

In [20]:
```
# Copy X, replace nonsquares in Y, and display the requested results.

Y = X.copy()
squares = np.arange(1, 11) ** 2
mask = ~np.isin(Y, squares)
Y[mask] = 0

print(X)
print(Y)
```

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
[[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
 [  0   0   0   0   0  36   0   0   0   0]
 [  0   0   0   0   0   0   0   0  49   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [  0   0   0  64   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [ 81   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0 100]]
```

Your explanation: Using Y = X simply leads to Y referring to the same object as X, so any changes made to Y, like replacing nonsquares with zero, would also change X. An independent copy needs `X.copy()`, which creates a new array with its own memory.

NP.5: Tile a checkerboard

Start with the integer array `[[1, 0], [0, 1]]`. Use `np.tile` to repeat it into an **8 × 8** array named `checkers`. Display it and its shape.

The first row should be `[1 0 1 0 1 0 1 0]` and the second `[0 1 0 1 0 1 0 1]`, with those rows alternating.

In [21]:
```
# Create and display checkers and its shape.

base = np.array([[1, 0], [0, 1]])
checkers = np.tile(base, (4, 4))
```

```
print(checkers)
print(checkers.shape)
```

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
(8, 8)
```

NP.6: An elementwise unit conversion

Convert every value of `f_temp` from Fahrenheit to Celsius: subtract 32, then multiply by 5/9. Round to one decimal place and save the result as `c_temp`. Display it without modifying `f_temp`.

The result retains shape `(5, 6)`. Its first value should be `21.1` and its last value `37.2`.

In [22]:
```
# Supplied input; do not modify this cell.
f_temp = np.arange(70, 100).reshape((5, 6))
```

In [23]:
```
# Create and display c_temp.

c_temp = np.round(((f_temp) - 32) * (5/9), 1)
print(c_temp)
```

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 [24.4 25.  25.6 26.1 26.7 27.2]
 [27.8 28.3 28.9 29.4 30.  30.6]
 [31.1 31.7 32.2 32.8 33.3 33.9]
 [34.4 35.  35.6 36.1 36.7 37.2]]
```

NP.7: Standardize each column

Convert the supplied array `x` to columnwise z-scores, stored in `z`: subtract each column's mean and divide by that column's SD. Use **`ddof=0`** for both standardizing and checking the result. These columns all have nonzero SD.

Use broadcasting, without loops. Display `z`, its column means, and its column SDs. The means should be approximately zero and the SDs approximately one. Do not round `z` before checking; values such as `1e-16` can be floating-point rounding rather than an error.

Understanding: Which `axis` produces one summary per column, and what is the shape of that summary? Explain how it broadcasts against `x` of shape `(10, 3)`.

In [24]:
```
# Supplied input; do not modify this cell.
rng = np.random.default_rng(seed=100)
x = rng.integers(0, 100, size=30).reshape(10, 3)
```

In [25]:
```
# Create z and display it, its column means, and its column SDs.

col_means = x.mean(axis=0)
col_sds = x.std(axis=0, ddof=0)

z = (x - col_means) / col_sds

print(z)
print(z.mean(axis=0))
print(z.std(axis=0, ddof=0))
```

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
 [-1.00437562 -1.26512499  0.46033272]
 [ 1.34064235  1.36446505  0.50143385]
 [-1.13711249  0.92620005  1.85777132]
 [ 1.07516861 -1.38199565  0.87134407]
 [ 0.19025617 -0.85607765  0.13152363]
 [ 1.38488797  0.60480571 -0.77270135]
 [-1.53532308  0.42949971 -1.34811724]
 [-0.38493691  0.28341137  0.50143385]]
[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

Your explanation: Axis 0 produces one summary per column and the shape is (3,), meaning that the broadcasting treats (3,) as a row that is repeated across all rows, so each x is standardized.

NP.8: Scale each column to 0–10

For the supplied `x`, linearly rescale **each column separately** so that its minimum becomes 0 and its maximum becomes 10. Round to two decimal places and store the result in `y`. Display `y` and its column minima and maxima.

These columns all have different minima and maxima. In the course environment, the first two rows are:

```
[[6.67 8.83 0.  ]
 [4.09 0.85 2.  ]]
```

Understanding: How is this transformation different from z-score standardization? What would happen to your formula if a column were constant? You do not need to implement a special rule for constant columns.

In [26]:
```
# Supplied input; do not modify this cell. Same values as NP.7.
rng = np.random.default_rng(seed=100)
x = rng.integers(0, 100, size=30).reshape(10, 3)
```

In [27]:
```
# Create y and display it, its column minima, and its column maxima.

col_mins = x.min(axis=0)
col_maxs = x.max(axis=0)

y = np.round((x - col_mins) / (col_maxs - col_mins) * 10, 2)

print(y)
print(y.min(axis=0))
print(y.max(axis=0))
```

```
[[ 6.67  8.83  0.  ]
 [ 4.09  0.85  2.  ]
 [ 1.82  0.43  5.75]
 [ 9.85 10.    5.88]
 [ 1.36  8.4  10.  ]
 [ 8.94  0.    7.  ]
 [ 5.91  1.91  4.75]
 [10.    7.23  2.  ]
 [ 0.    6.6   0.25]
 [ 3.94  6.06  5.88]]
[0. 0. 0.]
[10. 10. 10.]
```

Your explanation: This rescaling is different from z-score standardization because it uses each column's minimum and range to force the values into a fixed interval (0 to 10), rather than using the mean and SD to give a mean of 0 and SD of 1. If a column were constant, its maximum and minimum would be equal, so the formula would divide by zero (the range), producing `nan` values for that column.

NP.9: Detect and replace NaN values

Make an independent copy of the supplied `x` named `x_filled`. Use a Boolean mask to replace its `NaN` entries with zero, leaving the original `x` unchanged. Display the original array, the filled array, and the number of entries replaced.

Expected filled array:

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```

Understanding: Why should you use `np.isnan(x)` instead of `x == np.nan`? Replacing missing values with zero here is an array-manipulation exercise. Give one reason it could distort an analysis if zero were not a justified value for a missing measurement.

In [28]:
```
# Supplied input; do not modify this cell.
rng = np.random.default_rng(seed=100)
x = rng.integers(0, 100, size=12).reshape(4, 3).astype(float)
row = np.array([1, 2, 3])
```

```
col = np.array([2, 1, 0])
x[row, col] = np.nan
```

In [29]:
```
# Create x_filled, replace its NaNs, and display the requested results.

x_filled = x.copy()
mask = np.isnan(x_filled)
x_filled[mask] = 0

print(x)
print(x_filled)
print(mask.sum())
```

```
[[76. 83. 12.]
 [59.  8. nan]
 [44. nan 58.]
 [nan 94. 59.]]
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
3
```

Your explanation: x == np.nan doesn't work because np.nan never equals anything, so it will always return false; therefore, np.isnan should be used for the Boolean mask instead. Replacing missing values with zeros in an analysis could result in the zeros being treated as real measurements, affecting many statistical measures such as means and standard deviations.